

Faculty of Engineering and Technology
Department of Computer Science
COMP333—DATABASE SYSTEMS

Final Report: Consolidated Phases 1–3

Bawabet Al-Quds Roastery — Integrated Management System

Prepared by:

Yahya Hasan 1242481 Section 5

Mohammed Quraan 1243436 Section 5

Instructor: Yousef Hassouneh

Date: June 21, 2026

Contents

1 Project Overview

1.1 System Description

Bawabet Al-Quds Roastery is a specialty coffee and goods management system for a multi-branch roastery business based in Ramallah, Palestine. The system supports two classes of end-users—*customers* and *staff*—and is implemented as a JavaFX desktop application communicating with a MySQL database through JDBC.

Customers may browse items, manage a persistent shopping cart (with search, sort, and type-filter functionality), place orders, and submit reviews for items belonging to delivered orders. Staff access is role-stratified into three tiers: *Super Admin* (branch 1, system-wide authority), *Branch Admin* (single-branch authority over employees and orders), and *Employee* (operational access).

1.2 Main Functionality

1. **Authentication** — login with name, email, and password; role-based redirection to customer home or admin panel.
2. **Individual Customer Registration** — inserts into `Person`, `Customer`, and `Individual`.
3. **Business Customer Registration** — additionally inserts a `Location` row and links through `Business`.
4. **Customer Profile Management** — view and update name, email, phone, delivery address, and password.
5. **Product Catalog** — browseable home screen with name-based search. Items are categorised as Coffee, Roasts, Spice, or Package.
6. **Cart Management** — persistent cart with search, sort (price/name), and type-filter; running total; branch-aware quantity tracking.
7. **Order Placement** — atomically converts the cart into an `Order` with `OrderItem` snapshots and a `Delivery` record; adjusts `BranchInventory`.
8. **Order History & Status** — customer view (own orders); admin view (branch or all); status transitions (*pending* → *delivered*).
9. **Reviews** — a customer may review an item only if at least one of their orders containing that item has reached *delivered* status.
10. **Super Admin Panel** — manage branches, register admins and employees to any branch, view all orders system-wide.
11. **Branch Admin Panel** — register employees to own branch, view branch orders only.
12. **Item & Package Management** — add, edit, or delete catalog items; compose packages from existing items; every item and package is assigned to a supplier (defaulting to *Bawabet Al-Quds*).
13. **Inventory Management** — branch inventory and warehouse stock tracking.

2 Entity–Relationship Diagram



2.1 Entity Sets

Entity	Primary Key	Description
Person	person_id	Supertype for all human actors. Stores name, email, password, and optional phone.
Customer	person_id	ISA Person. Discriminated by type (<i>individual</i> or <i>business</i>); optionally linked to a Location.
Individual	person_id	ISA Customer. Retail buyer subtype; no extra attributes.
Business	person_id	ISA Customer. Wholesale buyer; holds registration number, TAX id, and business type.
Employee	person_id	ISA Person. Assigned to one branch; stores role, salary, and hire date.
Admin	person_id	ISA Person. Manages a branch. branch_id = 1 identifies the Super Admin.
Branch	branch_id	Physical store location; referenced by Employees, Admins, Orders, and BranchInventory.
Location	location_id	Reusable address (street, city, zip) shared by Branch, Warehouse, and Customer.
Warehouse	warehouse_id	Central storage facility linked to a Location; stock tracked in WarehouseInventory.
Supplier	supplier_id	External vendor. Every Item must have a supplier; the default is <i>Bawabet Al-Quds</i> .
Item	item_id	Supertype for all sellable goods; discriminated by item.type (Coffee, Roasts, Spice, Package).
Coffee	item_id	ISA Item. Represents coffee products.
Roasts	item_id	ISA Item. Represents roasted goods.
Spice	item_id	ISA Item. Represents spices and herbs.
Package	package_id	ISA Item. A curated bundle of multiple Items sold as one unit.
Order	order_id	Finalized purchase; records customer, branch, date, status, and cached total.
Delivery	delivery_id	Tracks shipment of an order: address, delivery date, and status (<i>pending</i> or <i>delivered</i>).
Payment	payment_id	Financial settlement of an order: amount, method, and processing employee.
Review	review_id	Customer rating and comment for an item; only permitted after a <i>delivered</i> order.
Discount	discount_id	Promotional codes with a percentage discount, usage cap, and expiry date.

2.2 Relationship Sets

Relationship	Entities	Description	Card.
ISA (Person)	Person, {Customer, Employee, Admin}	Person ISA hierarchy; subtypes are disjoint at application level.	1:1
ISA (Customer)	Customer, {Individual, Business}	Customer sub-hierarchy; disjoint.	1:1
ISA (Item)	Item, {Coffee, Roasts, Spice, Package}	Item sub-hierarchy; discriminated by <code>item_type</code> .	1:1
located_at	Branch, Location	Each branch must have one location (total participation).	N:1
wh_located	Warehouse, Location	Each warehouse has one location.	N:1
biz_addr	Business, Location	Optional delivery address for a business customer.	N:1
manages	Admin, Branch	An admin manages one branch; branch 1 admin is Super Admin.	N:1
works_at	Employee, Branch	Employee assigned to exactly one branch (total participation).	N:1
supplies	Supplier, Item	A supplier provides items; every item must have a supplier.	1:N
stocks (BranchInv.)	Branch, Item	Quantity of an item stocked at a branch.	M:N
holds (WH Inv.)	Warehouse, Item	Quantity of an item stored in a warehouse.	M:N
in_cart (Cart)	Customer, Item	Items in a customer's cart; attrs: quantity, branch_id.	M:N
places	Customer, Order	A customer places zero or more orders.	1:N
includes (OrderItem)	Order, Item	Line items of an order with price snapshot; attrs: qty, unit_price.	M:N
contains (PkgItem)	Package, Item	Items bundled inside a package; attr: quantity.	M:N
has_delivery	Order, Delivery	One delivery record per order.	1:1
has_payment	Order, Payment	One or more payment records per order.	1:N
processed_by	Payment, Employee	Optional employee who processed the payment.	N:1
writes (Review)	Person, Item	Customer reviews an item; only after a delivered order.	M:N

3 Relational Schema

3.1 Core Identity

```
1 CREATE TABLE Location (  
2     location_id INT          AUTO_INCREMENT PRIMARY KEY,  
3     street      VARCHAR(100) NOT NULL,  
4     city        VARCHAR(45)  NOT NULL,  
5     zip         VARCHAR(45)  NOT NULL  
6 );  
7  
8 CREATE TABLE Branch (  
9     branch_id   INT          AUTO_INCREMENT PRIMARY KEY,  
10    branch_name  VARCHAR(45)  DEFAULT NULL,  
11    location_id  INT          NOT NULL,  
12    CONSTRAINT fk_branch_location  
13        FOREIGN KEY (location_id) REFERENCES Location(location_id)  
14 );
```

Listing 1: Location and Branch

```
1 CREATE TABLE Person (  
2     person_id INT          AUTO_INCREMENT PRIMARY KEY,  
3     name      VARCHAR(45)  NOT NULL,  
4     email     VARCHAR(45)  NOT NULL,  
5     password  VARCHAR(100) NOT NULL,  
6     phone     VARCHAR(45)  DEFAULT NULL,  
7     UNIQUE KEY uq_person_email      (email),  
8     UNIQUE KEY uq_person_name_email (name, email)  
9 );
```

Listing 2: Person supertype

3.2 Person ISA Subtypes

```
1 CREATE TABLE Customer (  
2     person_id INT          PRIMARY KEY,  
3     type      VARCHAR(45)  NOT NULL, -- 'individual' or 'business'  
4     location_id INT        DEFAULT NULL,  
5     CONSTRAINT fk_customer_person FOREIGN KEY (person_id)  
6         REFERENCES Person(person_id) ON DELETE CASCADE,  
7     CONSTRAINT fk_customer_location FOREIGN KEY (location_id)  
8         REFERENCES Location(location_id) ON DELETE SET NULL  
9 );  
10  
11 CREATE TABLE Individual (  
12     person_id INT PRIMARY KEY,  
13     CONSTRAINT fk_individual_customer  
14         FOREIGN KEY (person_id) REFERENCES Customer(person_id) ON  
15         DELETE CASCADE  
16 );  
17  
18 CREATE TABLE Business (  
19     person_id INT          PRIMARY KEY,  
20     registration_number VARCHAR(45) NOT NULL,  
21     TAX_id     VARCHAR(45)  NOT NULL,  
22     business_type VARCHAR(45)  NOT NULL,
```

```

22     CONSTRAINT fk_business_customer
23         FOREIGN KEY (person_id) REFERENCES Customer(person_id) ON
24         DELETE CASCADE
25 );

```

Listing 3: Customer ISA hierarchy

```

1  CREATE TABLE Employee (
2      person_id INT PRIMARY KEY,
3      role VARCHAR(45) NOT NULL,
4      salary DECIMAL(10,0) NOT NULL,
5      hire_date DATE NOT NULL,
6      branch_id INT NOT NULL,
7      CONSTRAINT fk_employee_person FOREIGN KEY (person_id)
8          REFERENCES Person(person_id) ON DELETE CASCADE,
9      CONSTRAINT fk_employee_branch FOREIGN KEY (branch_id)
10         REFERENCES Branch(branch_id)
11 );
12
13 CREATE TABLE Admin (
14     person_id INT PRIMARY KEY,
15     branch_id INT NOT NULL,
16     -- branch_id = 1 identifies the Super Admin
17     CONSTRAINT fk_admin_person FOREIGN KEY (person_id)
18         REFERENCES Person(person_id) ON DELETE CASCADE,
19     CONSTRAINT fk_admin_branch FOREIGN KEY (branch_id)
20         REFERENCES Branch(branch_id)
21 );

```

Listing 4: Employee and Admin ISA subtypes

3.3 Supplier and Items

```

1  CREATE TABLE Supplier (
2      supplier_id INT AUTO_INCREMENT PRIMARY KEY,
3      name VARCHAR(100) NOT NULL,
4      email VARCHAR(100) DEFAULT NULL,
5      phone VARCHAR(45) DEFAULT NULL
6  );
7
8  CREATE TABLE Item (
9      item_id INT AUTO_INCREMENT PRIMARY KEY,
10     name VARCHAR(100) NOT NULL DEFAULT '',
11     item_type VARCHAR(45) DEFAULT NULL,
12     price DECIMAL(10,0) DEFAULT NULL,
13     wholesale_price DECIMAL(10,0) DEFAULT NULL,
14     image_path VARCHAR(500) DEFAULT NULL,
15     supplier_id INT NOT NULL,
16     CONSTRAINT fk_item_supplier FOREIGN KEY (supplier_id)
17         REFERENCES Supplier(supplier_id) ON DELETE RESTRICT
18 );
19
20 -- Many-to-many: supplier can supply multiple items
21 CREATE TABLE SupplierItem (
22     supplier_id INT NOT NULL,
23     item_id INT NOT NULL,
24     supply_price DECIMAL(10,2) DEFAULT NULL,

```

```

25 PRIMARY KEY (supplier_id, item_id),
26 CONSTRAINT fk_si_supplier FOREIGN KEY (supplier_id)
27 REFERENCES Supplier(supplier_id) ON DELETE CASCADE,
28 CONSTRAINT fk_si_item FOREIGN KEY (item_id)
29 REFERENCES Item(item_id) ON DELETE CASCADE
30 );

```

Listing 5: Supplier and Item

3.4 Item ISA Subtypes

```

1 CREATE TABLE Coffee (
2     item_id INT PRIMARY KEY,
3     CONSTRAINT fk_coffee_item FOREIGN KEY (item_id)
4     REFERENCES Item(item_id) ON DELETE CASCADE
5 );
6
7 CREATE TABLE Roasts (
8     item_id INT PRIMARY KEY,
9     CONSTRAINT fk_roasts_item FOREIGN KEY (item_id)
10    REFERENCES Item(item_id) ON DELETE CASCADE
11 );
12
13 CREATE TABLE Spice (
14     item_id INT PRIMARY KEY,
15     CONSTRAINT fk_spice_item FOREIGN KEY (item_id)
16     REFERENCES Item(item_id) ON DELETE CASCADE
17 );
18
19 CREATE TABLE Package (
20     package_id INT PRIMARY KEY,
21     name VARCHAR(100) NOT NULL,
22     description TEXT DEFAULT NULL,
23     CONSTRAINT fk_package_item FOREIGN KEY (package_id)
24     REFERENCES Item(item_id) ON DELETE CASCADE
25 );
26
27 -- Items contained within a package
28 CREATE TABLE PackageItem (
29     package_id INT NOT NULL,
30     item_id INT NOT NULL,
31     quantity INT NOT NULL DEFAULT 1,
32     PRIMARY KEY (package_id, item_id),
33     CONSTRAINT fk_pkgi_package FOREIGN KEY (package_id)
34     REFERENCES Package(package_id) ON DELETE CASCADE,
35     CONSTRAINT fk_pkgi_item FOREIGN KEY (item_id)
36     REFERENCES Item(item_id)
37 );

```

Listing 6: Item subtypes

3.5 Cart and Orders

```

1 -- Persistent shopping cart; branch_id tracks which branch stock to
  reserve
2 CREATE TABLE Cart (

```

```

3  person_id INT NOT NULL,
4  item_id INT NOT NULL,
5  quantity DECIMAL(10,3) NOT NULL DEFAULT 1.000,
6  branch_id INT NOT NULL DEFAULT 0,
7  PRIMARY KEY (person_id, item_id),
8  CONSTRAINT fk_cart_customer FOREIGN KEY (person_id)
9  REFERENCES Customer(person_id) ON DELETE CASCADE,
10 CONSTRAINT fk_cart_item FOREIGN KEY (item_id)
11 REFERENCES Item(item_id) ON DELETE CASCADE
12 );
13
14 CREATE TABLE 'Order' (
15  order_id INT AUTO_INCREMENT PRIMARY KEY,
16  person_id INT NOT NULL,
17  branch_id INT DEFAULT NULL,
18  order_date DATETIME DEFAULT CURRENT_TIMESTAMP,
19  status VARCHAR(45) DEFAULT 'pending',
20  payment_method VARCHAR(45) DEFAULT NULL,
21  total DECIMAL(10,2) DEFAULT 0.00,
22  CONSTRAINT fk_order_person FOREIGN KEY (person_id)
23  REFERENCES Person(person_id) ON DELETE CASCADE,
24  CONSTRAINT fk_order_branch FOREIGN KEY (branch_id)
25  REFERENCES Branch(branch_id) ON DELETE SET NULL
26 );
27
28 -- Price snapshot at time of purchase
29 CREATE TABLE OrderItem (
30  order_id INT NOT NULL,
31  item_id INT NOT NULL,
32  quantity DECIMAL(10,3) NOT NULL DEFAULT 1.000,
33  unit_price DECIMAL(10,2) NOT NULL,
34  PRIMARY KEY (order_id, item_id),
35  CONSTRAINT fk_oi_order FOREIGN KEY (order_id)
36  REFERENCES 'Order'(order_id) ON DELETE CASCADE,
37  CONSTRAINT fk_oi_item FOREIGN KEY (item_id)
38  REFERENCES Item(item_id)
39 );

```

Listing 7: Cart, Order, and OrderItem

```

1  -- status is either 'pending' or 'delivered'
2  CREATE TABLE Delivery (
3  delivery_id INT AUTO_INCREMENT PRIMARY KEY,
4  order_id INT NOT NULL,
5  address VARCHAR(200) NOT NULL,
6  delivered_at DATETIME DEFAULT NULL,
7  status VARCHAR(45) DEFAULT 'pending',
8  CHECK (status IN ('pending', 'delivered')),
9  CONSTRAINT fk_delivery_order FOREIGN KEY (order_id)
10 REFERENCES 'Order'(order_id) ON DELETE CASCADE
11 );
12
13 CREATE TABLE Payment (
14  payment_id INT AUTO_INCREMENT PRIMARY KEY,
15  order_id INT NOT NULL,
16  amount DECIMAL(10,2) NOT NULL,
17  method VARCHAR(45) NOT NULL,
18  employee_id INT DEFAULT NULL,

```

```

19     paid_at      DATETIME          DEFAULT CURRENT_TIMESTAMP,
20     CONSTRAINT fk_payment_order    FOREIGN KEY (order_id)
21         REFERENCES 'Order'(order_id) ON DELETE CASCADE,
22     CONSTRAINT fk_payment_employee FOREIGN KEY (employee_id)
23         REFERENCES Employee(person_id) ON DELETE SET NULL
24 );

```

Listing 8: Delivery and Payment

3.6 Inventory

```

1  CREATE TABLE BranchInventory (
2      branch_id INT          NOT NULL,
3      item_id   INT          NOT NULL,
4      quantity  DECIMAL(10,3) NOT NULL DEFAULT 0.000,
5      PRIMARY KEY (branch_id, item_id),
6      CONSTRAINT fk_bi_branch FOREIGN KEY (branch_id)
7          REFERENCES Branch(branch_id) ON DELETE CASCADE,
8      CONSTRAINT fk_bi_item  FOREIGN KEY (item_id)
9          REFERENCES Item(item_id)    ON DELETE CASCADE
10 );
11
12 CREATE TABLE Warehouse (
13     warehouse_id INT          AUTO_INCREMENT PRIMARY KEY,
14     location_id  INT          NOT NULL,
15     name         VARCHAR(100) NOT NULL,
16     CONSTRAINT fk_warehouse_location FOREIGN KEY (location_id)
17         REFERENCES Location(location_id)
18 );
19
20 CREATE TABLE WarehouseInventory (
21     warehouse_id INT          NOT NULL,
22     item_id      INT          NOT NULL,
23     quantity     DECIMAL(10,3) NOT NULL DEFAULT 0.000,
24     PRIMARY KEY (warehouse_id, item_id),
25     CONSTRAINT fk_wi_warehouse FOREIGN KEY (warehouse_id)
26         REFERENCES Warehouse(warehouse_id) ON DELETE CASCADE,
27     CONSTRAINT fk_wi_item      FOREIGN KEY (item_id)
28         REFERENCES Item(item_id)          ON DELETE CASCADE
29 );

```

Listing 9: Branch Inventory and Warehouse

3.7 Reviews and Discounts

```

1  -- Only permitted when a delivered order exists for that customer and
   -- item
2  CREATE TABLE Review (
3      review_id INT          AUTO_INCREMENT PRIMARY KEY,
4      person_id INT          NOT NULL,
5      item_id   INT          NOT NULL,
6      rating    TINYINT      NOT NULL,
7      comment   TEXT         DEFAULT NULL,
8      review_date DATETIME    DEFAULT CURRENT_TIMESTAMP,
9      CONSTRAINT fk_review_person FOREIGN KEY (person_id)
10         REFERENCES Person(person_id) ON DELETE CASCADE,

```

```

11      CONSTRAINT fk_review_item FOREIGN KEY (item_id)
12      REFERENCES Item(item_id) ON DELETE CASCADE
13 );
14
15 CREATE TABLE Discount (
16     discount_id INT AUTO_INCREMENT PRIMARY KEY,
17     code VARCHAR(50) NOT NULL,
18     percent DECIMAL(5,2) NOT NULL,
19     valid_until DATE DEFAULT NULL,
20     max_uses INT DEFAULT NULL,
21     UNIQUE KEY uq_discount_code (code)
22 );

```

Listing 10: Review and Discount

3.8 Seed Data

```
1  -- Address and first branch must exist before any person is inserted
2  INSERT INTO Location (street, city, zip)
3      VALUES ('Main Street 1', 'Ramallah', '00970');
4  INSERT INTO Branch (branch_name, location_id)
5      VALUES ('Main Branch', LAST_INSERT_ID());
6
7  -- Default supplier (branch_id = 1 is always the Main Branch / Super
   Admin branch)
8  INSERT INTO Supplier (name, email, phone)
9      VALUES ('Bawabet Al-Quds', 'info@bawabet.com', '+970-2-000-0000');
10
11 -- Super Admin (branch_id = 1 confers system-wide authority)
12 INSERT INTO Person (name, email, password)
13     VALUES ('admin', 'admin@bawabet.com', 'admin123');
14 INSERT INTO Admin (person_id, branch_id)
15     VALUES (LAST_INSERT_ID(),
16             (SELECT branch_id FROM Branch WHERE branch_name = 'Main
               Branch'));
```

Listing 11: Initial seed data required for system startup

4 SQL Queries by Functionality

4.1 User Login

```
1 SELECT person_id
2 FROM Person
3 WHERE email = '<email>'
4 AND name = '<name>'
5 AND password = '<password>';
```

Listing 12: Authenticate a user by name, email, and password

4.2 Register Individual Customer

```
1 -- Step 1: insert person row
2 INSERT INTO Person (name, email, password, phone)
3 VALUES ('<name>', '<email>', '<password>', '<phone>');
4
5 -- Step 2: mark as customer
6 INSERT INTO Customer (person_id, type)
7 VALUES (<generated_id>, 'individual');
8
9 -- Step 3: insert individual subtype
10 INSERT INTO Individual (person_id) VALUES (<generated_id>);
```

Listing 13: Three-step individual customer registration

4.3 Register Business Customer

```
1 -- Step 1: insert delivery address
2 INSERT INTO Location (street, city, zip)
3 VALUES ('<street>', '<city>', '<zip>');
4
5 -- Step 2: insert person row
6 INSERT INTO Person (name, email, password, phone)
7 VALUES ('<name>', '<email>', '<password>', '<phone>');
8
9 -- Step 3: mark as business customer linked to address
10 INSERT INTO Customer (person_id, type, location_id)
11 VALUES (<person_id>, 'business', <location_id>);
12
13 -- Step 4: insert business-specific details
14 INSERT INTO Business (person_id, registration_number, TAX_id,
15 business_type)
16 VALUES (<person_id>, '<reg>', '<tax>', '<kind>');
```

Listing 14: Four-step business customer registration

4.4 Retrieve Customer Profile

```
1 SELECT P.name, P.email, C.type, P.phone, L.street AS address
2 FROM Person P
3 JOIN Customer C ON P.person_id = C.person_id
4 LEFT JOIN Location L ON C.location_id = L.location_id
5 WHERE P.person_id = <session_person_id>;
```

Listing 15: Fetch profile info with optional delivery address

4.5 Update Profile

```
1 UPDATE Person
2 SET     name  = '<name>',
3         email = '<email>',
4         phone = '<phone>'
5 WHERE  person_id = <session_person_id>;
```

Listing 16: Update name, email, and phone

```
1 -- If customer already has a location_id, update it
2 UPDATE Location
3 SET     street = '<address>'
4 WHERE  location_id = (SELECT location_id FROM Customer WHERE person_id
5                        = ?);
6
7 -- Otherwise, insert a new Location row and link it
8 INSERT INTO Location (street, city, zip) VALUES ('<address>', '-', '-')
9 ;
10 UPDATE Customer SET location_id = LAST_INSERT_ID() WHERE person_id = ?;
```

Listing 17: Upsert delivery address

```
1 -- Step 1: verify current password
2 SELECT password FROM Person WHERE person_id = <session_person_id>;
3
4 -- Step 2: update if verified
5 UPDATE Person SET password = '<new_password>'
6 WHERE  person_id = <session_person_id>;
```

Listing 18: Change password (verified against current password)

4.6 Cart: Search, Sort, and Filter

```
1 SELECT I.item_id, I.name, I.price, C.quantity,
2        I.item_type, I.image_path,
3        COALESCE(C.branch_id, 0) AS branch_id
4 FROM   Cart C
5 JOIN   Item I ON C.item_id = I.item_id
6 WHERE  C.person_id = <user_id>
7        -- optional name search:
8        AND I.name LIKE '%<keyword>%'
9        -- optional type filter:
10       AND LOWER(I.item_type) = LOWER('<type>') -- omit line for 'All'
11 -- optional sort (choose one):
12 ORDER BY I.price ASC; -- Price: Low to High
13 -- ORDER BY I.price DESC; -- Price: High to Low
14 -- ORDER BY I.name ASC; -- Name: A-Z
15 -- ORDER BY I.name DESC; -- Name: Z-A
```

Listing 19: Flexible cart query with optional search, filter, and sort

```

1 SELECT SUM(I.price * C.quantity) AS total
2 FROM   Cart C
3 JOIN   Item I ON C.item_id = I.item_id
4 WHERE  C.person_id = <user_id>;

```

Listing 20: Full cart total (unaffected by view filter)

4.7 Place Order

```

1 -- Step 1: snapshot cart items
2 SELECT C.item_id, C.quantity, COALESCE(C.branch_id, 0) AS branch_id,
3        I.price
4 FROM   Cart C
5 JOIN   Item I ON C.item_id = I.item_id
6 WHERE  C.person_id = <user_id>;
7
8 -- Step 2: create the Order row
9 INSERT INTO 'Order' (person_id, branch_id, total)
10    VALUES (<user_id>, <branch_id>, <computed_total>);
11
12 -- Step 3: insert one OrderItem per cart row
13 INSERT INTO OrderItem (order_id, item_id, quantity, unit_price)
14    VALUES (<order_id>, <item_id>, <qty>, <price>);
15 -- (repeat for each item)
16
17 -- Step 4: reduce branch inventory
18 UPDATE BranchInventory
19 SET    quantity = GREATEST(0.000, quantity - <qty>)
20 WHERE  branch_id = <branch_id> AND item_id = <item_id>;
21
22 -- Step 5: create delivery record (status defaults to 'pending')
23 INSERT INTO Delivery (order_id, address)
24    VALUES (<order_id>, '<customer_address>');
25
26 -- Step 6: clear the cart
27 DELETE FROM Cart WHERE person_id = <user_id>;

```

Listing 21: Atomic order placement from cart contents

4.8 View Order History

```

1 SELECT O.order_id, O.order_date, O.status, O.total,
2        OI.item_id, I.name AS item_name,
3        OI.quantity, OI.unit_price
4 FROM   'Order' O
5 JOIN   OrderItem OI ON O.order_id = OI.order_id
6 JOIN   Item I ON OI.item_id = I.item_id
7 WHERE  O.person_id = <user_id>
8 ORDER BY O.order_date DESC;

```

Listing 22: Customer order history with item details

```

1 SELECT O.order_id, O.person_id, P.name AS person_name,
2        O.order_date, O.status, O.total, O.branch_id
3 FROM   'Order' O
4 JOIN   Person P ON O.person_id = P.person_id

```

```

5 WHERE  O.branch_id = <branch_id>           -- omit for Super Admin
6 ORDER  BY O.order_date DESC;

```

Listing 23: Admin: all orders for a branch (Super Admin omits WHERE)

4.9 Submit a Review (Delivered Orders Only)

```

1 SELECT 1
2 FROM    OrderItem OI
3 JOIN    'Order' O ON OI.order_id = O.order_id
4 WHERE   O.person_id = <user_id>
5     AND OI.item_id = <item_id>
6     AND O.status = 'delivered'
7 LIMIT 1;

```

Listing 24: Guard: check that the customer has a delivered order for the item

```

1 SELECT 1
2 FROM    Review
3 WHERE   person_id = <user_id>
4     AND item_id = <item_id>
5 LIMIT 1;

```

Listing 25: Check for existing review (one per customer-item pair)

```

1 INSERT INTO Review (person_id, item_id, rating, comment)
2 VALUES (<user_id>, <item_id>, <rating>, '<comment>');

```

Listing 26: Insert a new review

4.10 Admin: Manage Items

```

1 SELECT I.item_id, I.name, I.price, I.wholesale_price,
2        I.item_type, I.image_path, I.supplier_id, S.name AS
           supplier_name
3 FROM    Item I
4 JOIN    Supplier S ON I.supplier_id = S.supplier_id
5 ORDER  BY I.name;

```

Listing 27: List all items for the admin panel

```

1 -- Step 1: insert the base Item row
2 INSERT INTO Item (name, item_type, price, wholesale_price, image_path,
3     supplier_id)
4     VALUES ('<name>', 'Coffee', <price>, <wholesale>, '<path>', <
5     supplier_id>);
6
7 -- Step 2: insert the subtype row
8 INSERT INTO Coffee (item_id) VALUES (LAST_INSERT_ID());

```

Listing 28: Add a new item (Coffee example)

```

1 UPDATE Item
2 SET    name           = '<name>',
3        price          = <price>,
4        wholesale_price = <wholesale>,

```

```

5      item_type      = '<type>',
6      image_path     = '<path>',
7      supplier_id    = <supplier_id>
8 WHERE item_id = <item_id>;

```

Listing 29: Update an existing item

4.11 Admin: Register Employee

```

1 -- Super Admin may specify any branch_id; Branch Admin uses their own
   branch
2 INSERT INTO Person (name, email, password)
3     VALUES ('<name>', '<email>', '<password>');
4
5 INSERT INTO Employee (person_id, role, salary, hire_date, branch_id)
6     VALUES (<generated_id>, '<role>', <salary>, '<YYYY-MM-DD>', <
   branch_id>);

```

Listing 30: Register a new employee (Branch Admin limited to own branch)

4.12 Super Admin: Register Branch Admin and Add Branch

```

1 INSERT INTO Person (name, email, password)
2     VALUES ('<name>', '<email>', '<password>');
3
4 INSERT INTO Admin (person_id, branch_id)
5     VALUES (<generated_id>, <branch_id>);

```

Listing 31: Register a branch admin (Super Admin only)

```

1 INSERT INTO Location (street, city, zip)
2     VALUES ('<street>', '<city>', '<zip>');
3
4 INSERT INTO Branch (branch_name, location_id)
5     VALUES ('<branch_name>', LAST_INSERT_ID());

```

Listing 32: Create a new branch (Super Admin only)

4.13 Inventory Management

```

1 SELECT I.item_id, I.name, I.item_type, BI.quantity
2 FROM   BranchInventory BI
3 JOIN   Item      I   ON BI.item_id = I.item_id
4 WHERE  BI.branch_id = <branch_id>
5 ORDER BY I.name;

```

Listing 33: Retrieve branch inventory for admin view

```

1 INSERT INTO BranchInventory (branch_id, item_id, quantity)
2     VALUES (<branch_id>, <item_id>, <qty>)
3 ON DUPLICATE KEY UPDATE quantity = quantity + <qty>;

```

Listing 34: Restock a branch item (upsert)

5 Normalization Analysis

This section demonstrates that every relation in the Bawabet Al-Quds schema satisfies **Third Normal Form (3NF)**. Recall the definitions used throughout:

- **1NF**: All attributes are atomic; no repeating groups.
- **2NF**: 1NF, plus every non-prime attribute is fully functionally dependent on *every* candidate key (no partial dependencies exist when the candidate key is composite).
- **3NF**: 2NF, plus no non-prime attribute is transitively dependent on any candidate key (i.e., for every non-trivial FD $X \rightarrow A$, either X is a superkey or A is a prime attribute).

5.1 First Normal Form (1NF)

All tables in the schema store only atomic, single-valued attributes. Composite or multi-valued information is stored in separate tables (e.g., a customer's cart items are rows in **Cart**, not a list inside **Customer**). Repeating groups are absent by design. Therefore, **all relations are in 1NF**.

5.2 Second Normal Form (2NF)

2NF violations only arise in relations with composite candidate keys when some non-key attribute depends on a strict subset of that composite key. The tables below have composite primary keys; the analysis confirms no partial dependencies exist.

Table	Composite PK	Non-key attributes
Cart	(person_id, item_id)	quantity, branch_id
OrderItem	(order_id, item_id)	quantity, unit_price
PackageItem	(package_id, item_id)	quantity
BranchInventory	(branch_id, item_id)	quantity
WarehouseInventory	(warehouse_id, item_id)	quantity
SupplierItem	(supplier_id, item_id)	supply_price

In every case the non-key attribute (**quantity** or **unit_price**) represents a fact about the *combination* of both key columns, not about either column alone. For example, **BranchInventory.quantity** is the stock of a specific item *at* a specific branch; it cannot be determined from **branch_id** or **item_id** alone. **All relations are in 2NF**.

5.3 Third Normal Form (3NF)

3NF is violated when a non-key attribute determines another non-key attribute (transitive dependency). The following analysis covers the relations most likely to harbour such a violation.

5.3.1 Relation: Person

Candidate keys: {person_id}, {email}, {name, email}

Functional dependencies:

$\text{person_id} \rightarrow \{\text{name}, \text{email}, \text{password}, \text{phone}\}$
 $\text{email} \rightarrow \{\text{person_id}, \text{name}, \text{password}, \text{phone}\}$

Every non-trivial FD has a superkey on the left-hand side. No transitive dependencies exist. **Person is in 3NF.**

5.3.2 Relation: Item

Candidate key: $\{\text{item_id}\}$

Functional dependencies:

$\text{item_id} \rightarrow \{\text{name}, \text{item_type}, \text{price}, \text{wholesale_price}, \text{image_path}, \text{supplier_id}\}$

One potential concern: could **supplier_id** transitively determine supplier attributes such as name, email, and phone? That concern is resolved by extracting **Supplier** into its own relation. Within **Item**, **supplier_id** is a foreign key only; all supplier attributes live exclusively in **Supplier**, so no transitive dependency exists inside **Item**. **Item is in 3NF.**

Prior violation (now corrected): in an earlier design iteration a **RoastBatch** table stored batch-level roasting details (roast level, origin, batch date) directly alongside **item_id**, which produced dependencies of the form $\text{item_id} \rightarrow \text{batch_id} \rightarrow \{\text{roast_level}, \text{origin}\}$ — a textbook transitive dependency. The **RoastBatch** table has been dropped; subtype tables (**Coffee**, **Roasts**) now store only the surrogate key, and detailed batch tracking can be added as a normalised sub-relation if required in a future phase.

5.3.3 Relation: Order

Candidate key: $\{\text{order_id}\}$

Functional dependencies:

$\text{order_id} \rightarrow \{\text{person_id}, \text{branch_id}, \text{order_date}, \text{status}, \text{payment_method}, \text{total}\}$

The stored **total** is technically derivable from **OrderItem**; however, it is retained as a *historical snapshot* that remains stable even when item prices change after the order is placed. No attribute determines another non-key attribute; no transitive dependency exists within this relation. **Order is in 3NF.**

5.3.4 Relation: Customer

Candidate key: $\{\text{person_id}\}$

Functional dependencies:

$\text{person_id} \rightarrow \{\text{type}, \text{location_id}\}$

location_id is a foreign key; the location's attributes (street, city, zip) live in the separate **Location** relation. There is therefore no path by which a non-key attribute of **Customer** determines another non-key attribute of **Customer**. **Customer is in 3NF.**

5.3.5 Relation: Business

Candidate key: {person_id}

Functional dependencies:

$$\text{person_id} \rightarrow \{\text{registration_number}, \text{TAX_id}, \text{business_type}\}$$

No functional dependency exists among the three non-key attributes (a business can change its registration number independently of its type). **Business is in 3NF.**

5.3.6 Relation: Branch

Candidate key: {branch_id}

Functional dependencies:

$$\text{branch_id} \rightarrow \{\text{branch_name}, \text{location_id}\}$$

Location details are kept in a separate **Location** row, eliminating any transitive path through street/city/zip. **Branch is in 3NF.**

5.3.7 Relation: Delivery

Candidate key: {delivery_id}

Functional dependencies:

$$\text{delivery_id} \rightarrow \{\text{order_id}, \text{address}, \text{delivered_at}, \text{status}\}$$

status is constrained to {*pending*, *delivered*} only; no employee link exists in this table. A potential concern: does **order_id** determine **address**, creating a transitive path $\text{delivery_id} \rightarrow \text{order_id} \rightarrow \text{address}$? It does not, because the delivery address is recorded *at the time of dispatch* and may differ from the customer's current location. **Delivery is in 3NF.**

5.4 Normalisation Summary

Relation	1NF	2NF	3NF
Person	✓	✓	✓
Customer	✓	✓	✓
Individual	✓	✓	✓
Business	✓	✓	✓
Employee	✓	✓	✓
Admin	✓	✓	✓
Branch	✓	✓	✓
Location	✓	✓	✓
Warehouse	✓	✓	✓
Supplier	✓	✓	✓
Item	✓	✓	✓
Coffee	✓	✓	✓
Roasts	✓	✓	✓
Spice	✓	✓	✓
Package	✓	✓	✓
SupplierItem	✓	✓	✓
PackageItem	✓	✓	✓
BranchInventory	✓	✓	✓
WarehouseInventory	✓	✓	✓
Cart	✓	✓	✓
Order	✓	✓	✓
OrderItem	✓	✓	✓
Delivery	✓	✓	✓
Payment	✓	✓	✓
Review	✓	✓	✓
Discount	✓	✓	✓

All twenty-six relations in the Bawabet Al-Quds schema satisfy Third Normal Form. The primary design decisions that enabled this outcome were: (1) extraction of **Supplier** into its own table to eliminate transitive supplier-attribute dependencies inside **Item**; (2) removal of the **RoastBatch** table, which had introduced a two-step transitive chain through a surrogate batch key; (3) use of surrogate auto-increment primary keys on every entity, ensuring all non-key attributes depend directly on the key.

6 Java Application Code

6.1 JDBC Connection

```
1 public class DBConnection {
2     private static final String URL =
3         "jdbc:mysql://localhost:3306/bawabet_db"
4         + "?useSSL=false&serverTimezone=UTC&allowPublicKeyRetrieval=
5             true";
6     private static final String USER      = "root";
7     private static final String PASSWORD = "p4ssw0rd";
8     private static Connection connection;
9
10    public static Connection getConnection() {
11        try {
12            if (connection == null || connection.isClosed())
13                connection = DriverManager.getConnection(URL, USER,
14                    PASSWORD);
15        } catch (SQLException e) { throw new RuntimeException(e); }
16        return connection;
17    }
18 }
```

Listing 35: Singleton JDBC connection — DBConnection.java

6.2 Authentication Logic

```
1 public static String login(SystemHandling sys,
2                             String name, String email, String password)
3     {
4         if (name.isEmpty() || email.isEmpty() || password.isEmpty())
5             return "empty";
6         try {
7             Statement stmt = sys.getConn().createStatement();
8             ResultSet rs = stmt.executeQuery(
9                 "SELECT person_id FROM Person WHERE email = '" + email +
10                 "' AND name = '" + name + "' AND password = '" + password +
11                 "'");
12             if (rs.next()) {
13                 sys.setCurrentUser(rs.getInt("person_id"), name, email);
14                 return "ok";
15             }
16             return "not_found";
17         } catch (SQLException ex) { return "error"; }
18     }
```

Listing 36: Login — Auth_Logic.java

6.3 Cart Query with Sort and Filter

```
1 private static List<CartItemRow> query(Connection conn, int userId,
2     String search, String sort, String filterType) {
3     StringBuilder sql = new StringBuilder(
4         "SELECT I.item_id, I.name, I.price, C.quantity, " +
5         "      I.item_type, I.image_path, " +
6         "      COALESCE(C.branch_id, 0) AS branch_id " +
```

```

7      "FROM Cart C JOIN Item I ON C.item_id = I.item_id " +
8      "WHERE C.person_id = ?");
9
10     if (search != null && !search.isBlank())
11         sql.append(" AND I.name LIKE ?");
12     if (filterType != null && !filterType.isBlank()
13         && !"All".equalsIgnoreCase(filterType))
14         sql.append(" AND LOWER(I.item_type) = LOWER(?)");
15
16     switch (sort != null ? sort : "") {
17     case "Price: Low to High" -> sql.append(" ORDER BY I.price ASC
18         ");
19     case "Price: High to Low" -> sql.append(" ORDER BY I.price
20         DESC");
21     case "Name: A-Z" -> sql.append(" ORDER BY I.name ASC
22         ");
23     case "Name: Z-A" -> sql.append(" ORDER BY I.name DESC
24         ");
25     }
26     // bind parameters and execute...
27 }

```

Listing 37: Dynamic cart query — Cart_Logic.java

6.4 Review Guard

```

1 public static boolean hasPurchased(Connection conn, int personId, int
   itemId) {
2     try {
3         PreparedStatement ps = conn.prepareStatement(
4             "SELECT 1 FROM OrderItem OI " +
5             "JOIN 'Order' O ON OI.order_id = O.order_id " +
6             "WHERE O.person_id = ? AND OI.item_id = ? " +
7             " AND O.status = 'delivered' LIMIT 1");
8         ps.setInt(1, personId);
9         ps.setInt(2, itemId);
10        return ps.executeQuery().next();
11    } catch (SQLException ex) { return false; }
12 }

```

Listing 38: Delivered-order check before allowing a review — OrderDAO.java

6.5 Three-Tier Admin Check

```

1 // A person whose Admin row points to branch_id = 1 is the Super Admin
2 public static boolean isSuperAdmin(Connection conn, int personId) {
3     try {
4         PreparedStatement ps = conn.prepareStatement(
5             "SELECT 1 FROM Admin WHERE person_id = ? AND branch_id = 1
6             LIMIT 1");
7         ps.setInt(1, personId);
8         return ps.executeQuery().next();
9     } catch (SQLException ex) { return false; }
10 }

```

Listing 39: Super Admin identification — Admin_Logic.java